ENCODERS AND AUTOENCODERS

Concepts and TensorFlow Implementation

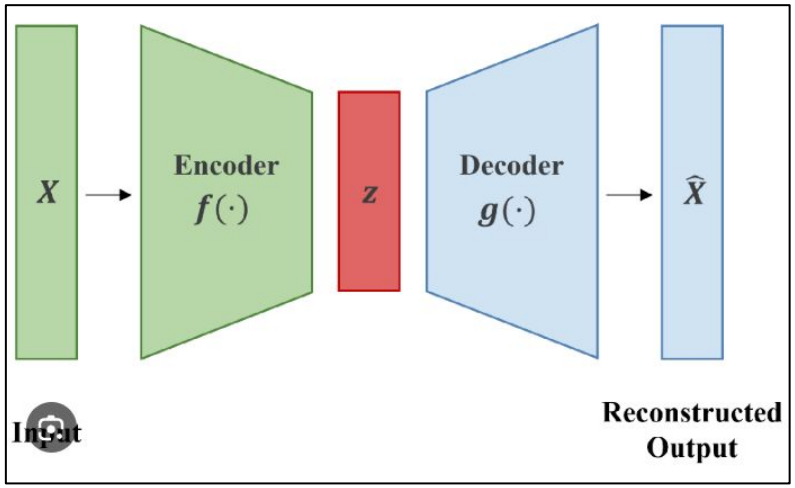
Paulo Vieira

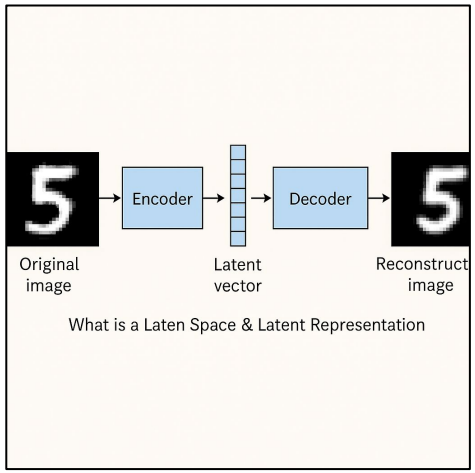
What is Latent Space & Latent Representation?

- A **latent space** is a compressed, abstract feature space learned by a model.

- **Latent representation** is a vector (set of values) that encodes essential features of the input.
- Captures **underlying patterns**, not raw data.
- Produced by the encoder, used by the decoder for reconstruction.
- Useful for **clustering**, **visualization**, and **semantic similarity**.

Original image → Encoder → **Latent vector** → Decoder → Reconstructed image





What is an Encoder?

- A neural-network|convolutional-model that compresses input data.
- Used for feature extraction and dimensionality reduction.
- Outputs a latent vector representation.
- latent vector representation is a compressed, often abstract, numerical representation of data
- It captures the underlying structure and essential features of the input data in a lower-dimensional space, allowing for more efficient processing and analysis.

What is Decoder?

- A **decoder** is a neural-network|convolutional-model that **reconstructs original input** from a compressed (latent) representation.
- Works as the **second half** of an autoencoder (Encoder → Decoder).
- Takes latent vector (encoding) and **maps it back** to the input space.
- Learns to reverse the compression done by the encoder.
- Often uses **mirrored architecture** of the encoder.
- In sequence models (like NLP), decoders also **generate sequences** based on context or attention (e.g., in Transformers).

What is an Autoencoder?

- Autoencoder = Encoder + Decoder
- Learns to reconstruct input from compressed form.
- Objective: minimize $||x - \hat{x}||$

Autoencoder Architecture

- Symmetrical: Encoder, Bottleneck, Decoder

  Bottleneck Layer (Definition)

  - The **narrowest point** in the autoencoder architecture.
  - It forces the model to **compress the input data** into a smaller, dense representation — the **latent vector**.
  - This compression encourages the model to learn only the **most important features** of the input.

- supondo a rede

```
Entrada → 128 → 64 → 16 → 64 → 128 → Saída
                     ↑
                Bottleneck
```

- Can be fully connected, convolutional, or recurrent.
- Latent space captures compressed data.

Autoencoder Architecture

- Symmetrical: Encoder, Bottleneck, Decoder
- Can be fully connected, convolutional, or recurrent.

In neural networks, the encoder and decoder components can be built using different types of architectures depending on the task.

A **fully connected (dense)** network connects every neuron from one layer to the next, making it suitable for general-purpose encoding of flattened data like tabular inputs or images (after reshaping).

A **convolutional** network, on the other hand, is ideal for image and spatial data — it uses filters that scan across the input, capturing local patterns like edges or textures with fewer parameters.

recurrent networks are designed for sequence data, such as text or time series, where information from previous steps needs to influence the current output — using structures like LSTM or GRU to maintain temporal context. Each of these architectures defines how the model learns and represents data in the latent space.

- Latent space captures compressed data.

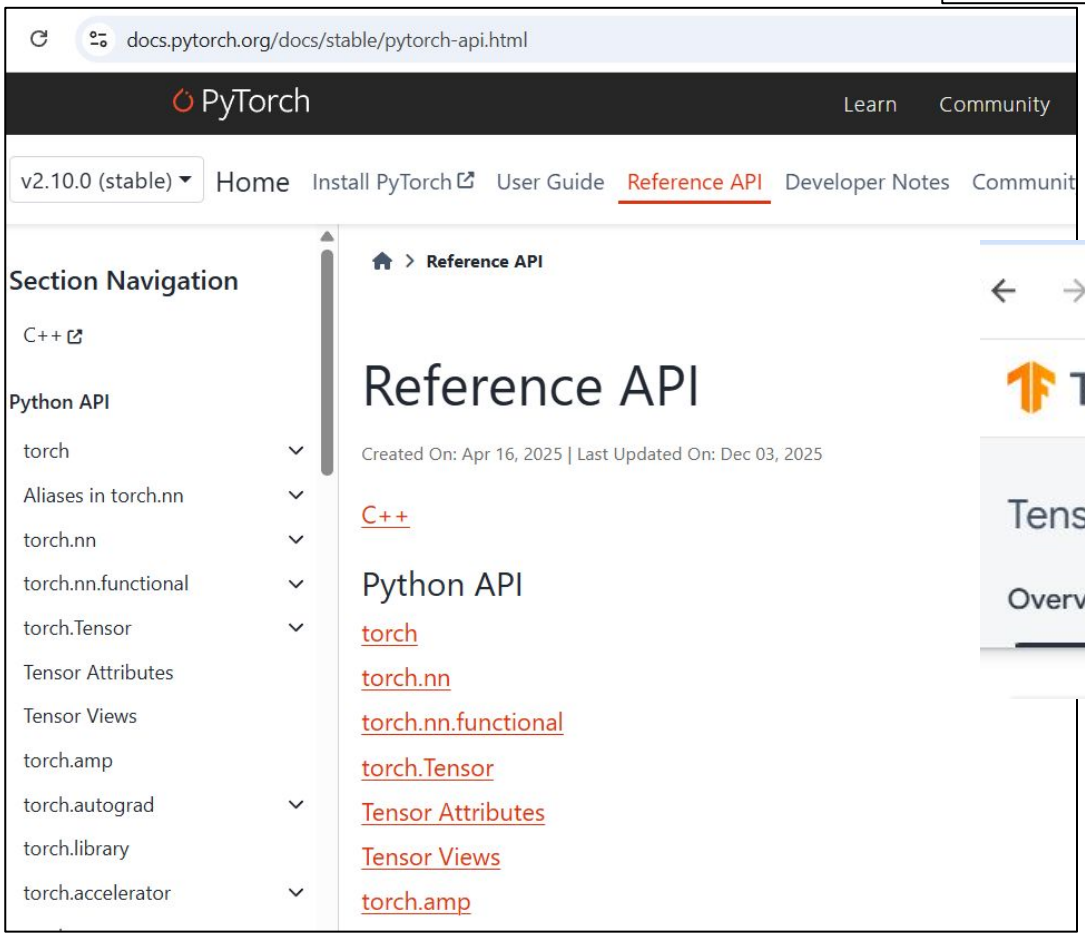
Diferença entre as redes neuronais

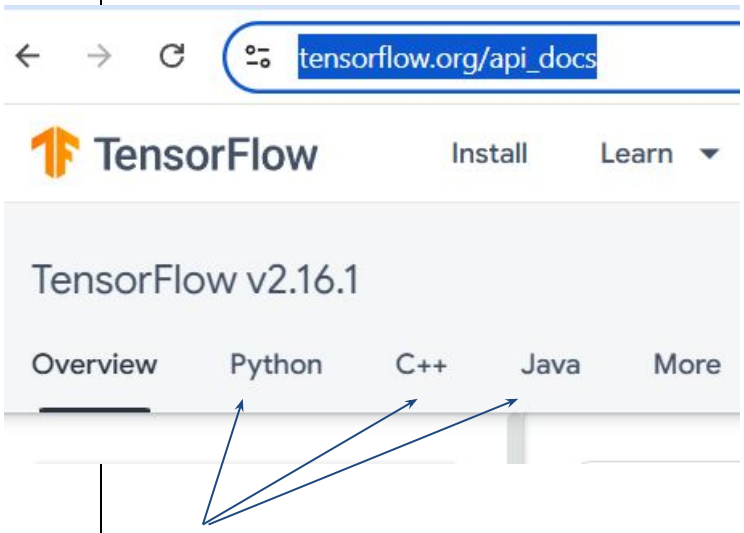
Característica	Fully Connected (Dense)	CNN (Convolutional)	RNN (Recurrent)
Estrutura	Todos os neurónios ligados entre si	Usa filtros locais (convoluções)	Ligações com memória (feedback)
Tipo de dados	Dados tabulares / vetores	Imagens / dados espaciais	Sequências / dados temporais
Relação entre dados	Não considera posição	Considera posição espacial	Considera ordem temporal
Memória	X Não tem	X Não tem	Tem memória (estado interno)
Operação principal	Multiplicação matricial	Convolução + pooling	Recorrência (estado anterior)
Nº de parâmetros	Alto	Mais baixo (partilha pesos)	Médio
Exemplo típico	Preço de casa	Classificar imagens	Prever palavras
Vantagem	Simples e geral	Muito eficiente em imagens	Capta dependências no tempo
Limitação	Não capta estrutura	Não capta sequência temporal	Difícil de treinar (problemas de memória longa)

exemplos sem treino em pytorch e tensorflow

https://docs.pytorch.org/docs/stable/pytorch-api.html

https://www.tensorflow.org/api_docs





exemplo (sem treino): prever o preço de uma casa

Fully Connected (Dense)

Exemplo: Prever preço de uma casa

Input:

- área (m²)
- número de quartos
- idade da casa
- localização (codificada)

Exemplo: [120,3,10,2]

O que a rede faz:

- Liga todos os inputs a todos os neurónios
- Aprende relações tipo:
  - mais área → preço maior
  - casa antiga → preço menor

Output: preço estimado

Exemplo: 250000€

Intuição: Não há “estrutura espacial” nem “ordem temporal” → só números → **Fully Connected é ideal**

exemplo (sem treino): prever o preço de uma casa

PyTorch

```
import torch
import torch.nn as nn

# Definir modelo
class DenseModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Sequential(
            nn.Linear(4, 16),  # 4 inputs → 16 neurónios
            nn.ReLU(),
            nn.Linear(16, 8),
            nn.ReLU(),
            nn.Linear(8, 1)   # output: preço
        )

    def forward(self, x):
        return self.fc(x)

# Criar modelo
model = DenseModel()

# Exemplo de input: [area, quartos, idade, localização]
x = torch.tensor([[120.0, 3.0, 10.0, 2.0]])

# Previsão
output = model(x)
print(output)
```

TensorFlow
https://www.tensorflow.org
TensorFlow

```
import tensorflow as tf
from tensorflow.keras import layers,
models

# Criar modelo
model = models.Sequential([
    layers.Dense(16, activation='relu',
input_shape=(4,)),
    layers.Dense(8, activation='relu'),
    layers.Dense(1)  # output: preço
])

# Exemplo de input
x = tf.constant([[120.0, 3.0, 10.0, 2.0]])

# Previsão
output = model(x)
print(output)
```

exemplo (sem treino): classificar imagem (gato vs cão)

CNN (Convolutional)

Exemplo: Classificar imagens de gatos vs cães

Input:

- imagem (ex: 64x64 pixels)

O que a rede faz:

- aplica filtros:
  - deteta bordas
  - depois formas (orelhas, olhos)
  - depois padrões mais complexos

Output:

- “gato” ou “cão”

Intuição

A CNN aprende coisas tipo:

- “tem orelhas pontiagudas?”
- “tem focinho comprido?”

Usa informação **local da imagem**

exemplo (sem treino): classificar imagem (gato vs cão)

PyTorch

```
import torch
import torch.nn as nn

class CNNModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(1, 8, kernel_size=3),  # 1 canal → 8 filtros
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(8, 16, kernel_size=3),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )

        self.fc = nn.Sequential(
            nn.Linear(16 * 5 * 5, 64),  # depende size da  imagem
            nn.ReLU(),
            nn.Linear(64, 2)  # 2 classes (gato/cão)
        )

    def forward(self, x):
        x = self.conv(x)
        x = x.view(x.size(0), -1)  # achatar
        x = self.fc(x)
        return x

# Criar modelo
model = CNNModel()
# Exemplo: imagem 28x28 (tipo MNIST)
x = torch.randn(1, 1, 28, 28)
output = model(x)
print(output)
```

TensorFlow
https://www.tensorflow.org
TensorFlow

```
import tensorflow as tf
from tensorflow.keras import layers, models

# Criar modelo
model = models.Sequential([
    layers.Conv2D(8, (3,3), activation='relu',
input_shape=(28, 28, 1)),
    layers.MaxPooling2D((2,2)),

    layers.Conv2D(16, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),

    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(2)  # 2 classes
])

# Exemplo de input
x = tf.random.normal((1, 28, 28, 1))

# Previsão
output = model(x)
print(output)
```

Exemplo (sem treino): prever próxima palavra

RNN (Recurrent)

Exemplo: Prever próxima palavra numa frase

Input: Frase: "Hoje está muito ___"

O que a rede faz:

- lê palavra a palavra:
  - "Hoje" → guarda contexto
  - "está" → atualiza memória
  - "muito" → usa contexto acumulado

Output: “frio” (ou “calor”, etc.)

Intuição

A rede:

- lembra-se do passado
- usa contexto da frase

Exemplo (sem treino): prever próxima palavra

```
import torch
import torch.nn as nn

class RNNModel(nn.Module):
  def __init__(self):
      super().__init__()
      self.rnn = nn.RNN(input_size=10, hidden_size=20, batch_first=True)
      self.fc = nn.Linear(20, 5)  # 5 palavras possíveis

  def forward(self, x):
      out, hidden = self.rnn(x)
      out = self.fc(out[:, -1, :])  # última saída
      return out

# Criar modelo
model = RNNModel()

# Exemplo: sequência de 3 palavras (cada uma com 10 features)
x = torch.randn(1, 3, 10)

output = model(x)
print(output)
```

TensorFlow
https://www.tensorflow.org
TensorFlow

```
import tensorflow as tf
from tensorflow.keras import layers, models

# Criar modelo
model = models.Sequential([
layers.SimpleRNN(20, input_shape=(3, 10)),
layers.Dense(5) # 5 classes possíveis
])

# Exemplo de input (batch, sequência,
features)
x = tf.random.normal((1, 3, 10))

# Previsão
output = model(x)
print(output)
```

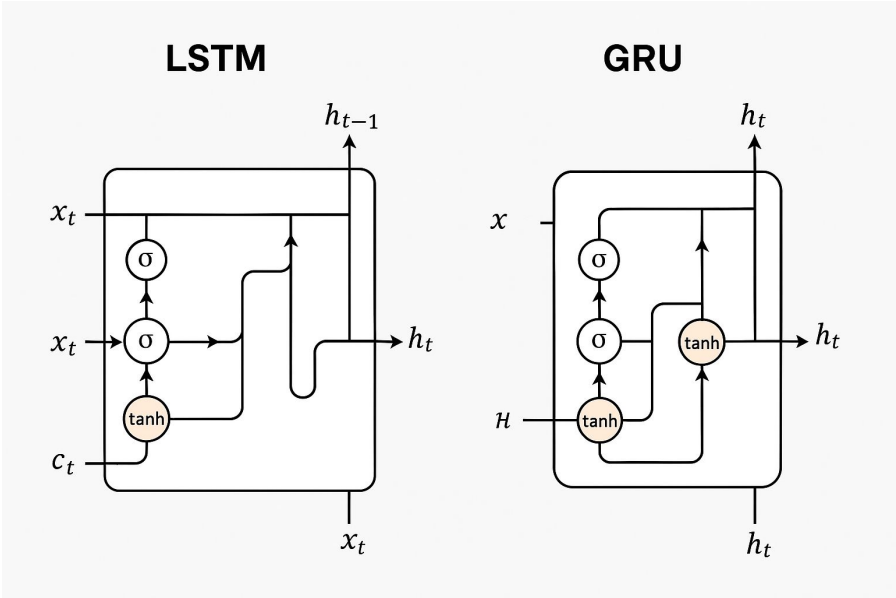
Os exemplos e as arquitecturas das redes usadas

Tipo	Exemplo	Porque usar
Fully Connected	Preço de casa	Dados sem estrutura
CNN	Imagens (gato/cão)	Estrutura espacial
RNN	Texto / séries temporais	Ordem importa

Autoencoder Architecture

Aspect	LSTM (Long Short-Term Memory)	GRU (Gated Recurrent Unit)
Purpose	Captures long-term dependencies in sequential data.	Captures long-term dependencies with a simpler structure than LSTM.
Structure	Uses three gates (input, forget, and output) to control information flow.	Uses two gates (update and reset) for information control.
Encoder Role	Encodes sequential input data into a compressed latent vector.	Encodes sequential input data into a compressed latent vector.
Decoder Role	Reconstructs the sequential data from the latent vector.	Reconstructs the sequential data from the latent vector.
Key Benefit	Better at handling long-term dependencies with a more complex gating system.	Computationally simpler and efficient while still capturing dependencies.
Applications	Time-series forecasting, anomaly detection, sequence generation.	Time-series forecasting, anomaly detection, sequence generation.
Main Advantage in Autoencoders	Effective in learning sequential data representations with temporal context.	Efficient learning of sequential data representations with fewer parameters.

Autoencoder Architecture



This image depicts the architecture of a **Long Short-Term Memory (LSTM)** network, a type of recurrent neural network (RNN) designed to model sequential data while mitigating the vanishing gradient problem.

The diagram likely illustrates key components such as the **hidden state (h_t)**, **cell state (c_t)**, and **input (x_t)**, with arrows or connections showing how information flows through time steps.

The recurrence of htht and x_t suggests an emphasis on the iterative nature of LSTMs, where past information (via h_{t−1}) and current input (x_t) are combined to update the cell state and produce a new hidden state.

The notation ctct highlights the LSTM's gating mechanisms (forget, input, and output gates), which regulate long-term dependencies.

The minimalist, text-heavy format implies a technical audience familiar with RNNs, focusing on the core mechanics rather than visual flair. If graphical elements (e.g., gates, activation functions) are omitted, the analysis may assume prior knowledge of LSTM internals.

Note: A more detailed breakdown would require seeing the actual connections or labels for gates (e.g., f_t, i_t, o_t).

LSTM- Manual em tensorflow

```
import tensorflow as tf
from tensorflow.keras import layers

class ManualLSTMCell(layers.Layer):
    def __init__(self, hidden_size):
        super().__init__()
        self.hidden_size = hidden_size

    def build(self, input_shape):
        input_size = input_shape[-1]

        # Pesos para todas as gates (input + hidden)
        self.W = self.add_weight(
            shape=(input_size + self.hidden_size,
4 * self.hidden_size),
            initializer='glorot_uniform',
            trainable=True
        )

        self.b = self.add_weight(
            shape=(4 * self.hidden_size,),
            initializer='zeros',
            trainable=True
        )
```

```
class ManualLSTMCell(layers.Layer):
    ....
    def call(self, x, states):
        h_prev, c_prev = states

        # juntar input + estado anterior
        combined = tf.concat([x, h_prev], axis=1)

        gates = tf.matmul(combined, self.W) + self.b

        # dividir em 4 partes
        f, i, o, g = tf.split(gates,
    num_or_size_splits=4, axis=1)

        #  Gates
        f = tf.sigmoid(f)   # Forget gate
        i = tf.sigmoid(i)   # Input gate
        o = tf.sigmoid(o)   # Output gate
        g = tf.tanh(g)      # Candidate

        #  Atualizar memória
        c = f * c_prev + i * g

        #   Output
        h = o * tf.tanh(c)

        return h, [h, c]
```

LSTM- codificação Manual em tensorflow

como usar a célula numa sequencia

```
# Criar célula
cell = ManualLSTMCell(hidden_size=16)

# RNN com a célula manual
rnn = tf.keras.layers.RNN(cell)

# Input: (batch, sequência, features)
x = tf.random.normal((1, 5, 10))

# Forward
output = rnn(x)

print(output)
```

Estrutura: (batch, sequência, features)
Em TensorFlow (Keras), uma LSTM recebe dados no formato:
(batch, timesteps, features)

O que é uma célula LSTM?

É a unidade básica que:

- recebe:
  - input atual x_t
  - estado anterior h_{t−1}, c_{t−1}
- devolve:
  - novo estado h_t, c_t

Como funciona no tempo:

O TensorFlow faz isto automaticamente:

t=1 → chama a célula
t=2 → chama a célula
t=3 → chama a célula
...

A **mesma célula** é usada em todos os passos da sequência

batch=1: Número de exemplos processados ao mesmo tempo

- aqui há **1 sequência**
- podias por exemplo ter: (32, 5, 10) e teria 32 sequências de uma vez

2. Sequência = 5 (timesteps). Número de passos no tempo
Exemplo: t1, t2, t3, t4, t5
Podem ser: palavras numa frase, dias numa série temporal, frames de vídeo

3. Features = 10. Número de características em cada passo
Exemplo no tempo t1:
[0.2, -1.3, 0.7, ..., 10 valores] Pode representar: embedding de palavra, sensores, variáveis financeiras

treinar a LSTM- codificação Manual em tensorflow

```
model = models.Sequential([
layers.RNN(cell),
layers.Dense(1)
])

# --- COMPILAR ---
model.compile(
optimizer='adam',
loss='mse'
)

# --- DADOS (exemplo simples) ---
x = tf.random.normal((100, 5, 1)) # 100 exemplos
y = tf.reduce_sum(x, axis=1) # objetivo: somar sequência

# --- TREINO ---
model.fit(x, y, epochs=5)
model.save("meu_modelo.h5")
```

substituir a parte do código anterior a amarelo pelo código apontado indicado

```
# Input: (batch, sequência, features)
x = tf.random.normal((1, 5, 10))

# Forward
output = rnn(x)

print(output)
```

Estrutura: (batch, sequê
Em TensorFlow (Keras), u
dados no formato:
(batch, timesteps, features

Isto guarda:

- arquitetura
- pesos
- optimizer

avaliar o LSTM- codificação Manual em tensorflow

```
from tensorflow.keras.models import load_model
# tens de redefinir a classe ManualLSTMCell tb aqui
class ManualLSTMCell(tf.keras.layers.Layer):
    def __init__(self, hidden_size):
        super().__init__()
        self.hidden_size = hidden_size

    def build(self, input_shape):
        input_size = input_shape[-1]
        self.W = self.add_weight(
            shape=(input_size + self.hidden_size, 4 * self.hidden_size),
            initializer='glorot_uniform', # inicialização aleatoria dos pesos
            trainable=True  # o parametros vai ser aprendido durante o treino
        )
        self.b = self.add_weight(
            shape=(4 * self.hidden_size,),
            initializer='zeros',
            trainable=True
        )

    def call(self, x, states):
        h_prev, c_prev = states
        combined = tf.concat([x, h_prev], axis=1)
        gates = tf.matmul(combined, self.W) + self.b
        f, i, o, g = tf.split(gates, 4, axis=1)
        f = tf.sigmoid(f)
        i = tf.sigmoid(i)
        o = tf.sigmoid(o)
        g = tf.tanh(g)
        c = f * c_prev + i * g
        h = o * tf.tanh(c)
        return h, [h, c]

# carregar modelo com custom_objects
model = load_model("meu_modelo.h5", custom_objects={
    "ManualLSTMCell": ManualLSTMCell
})
```

Novo script:

- redefinir classes customizadas
- carregar com custom_objects
- avaliar

```
import tensorflow as tf
from tensorflow.keras.models import
load_model

# carregar modelo
model = load_model("meu_modelo.h5")

# ver estrutura
model.summary()

# criar dados de teste (exemplo)
x_test = tf.random.normal((10, 5, 1))
y_test = tf.reduce_sum(x_test, axis=1)

# avaliar
loss = model.evaluate(x_test, y_test)

print("Loss:", loss)
```

Applications of Autoencoders

- Dimensionality reduction
- Denoising
- Anomaly detection
- Compression and generative models

Simple Autoencoder in TensorFlow

- from tensorflow.keras import layers, models
- encoder = Sequential([...])
- decoder = Sequential([...])
- autoencoder = Sequential([encoder, decoder])

Decoder Implementation

- Input shape matches encoder output (latent space)
- Final output shape matches input (e.g., 784 for MNIST)
- Use sigmoid for normalized pixel values

Training the Autoencoder

- Load and normalize dataset (e.g., MNIST)
- autoencoder.compile(optimizer='adam', loss='mse')
- autoencoder.fit(x_train, x_train, ...)

Using Encoder for Feature Extraction

- encoded_imgs = encoder.predict(x_test)
- Latent features can be used for ML tasks
- Useful in clustering, visualization, etc.

Summary

- Encoders compress input data to essential features.
- Autoencoders reconstruct input from these features.
- TensorFlow enables rapid prototyping.
- Supports unsupervised learning and generative models.

FIM